Attention, Transformers, and Fusion

ELG 5218 - Uncertainty Evaluation in Engineering Measurements and Machine Learning

Miodrag Bolic

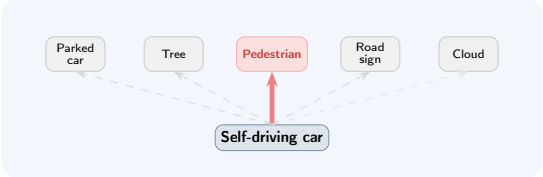
University of Ottawa

March 25, 2026

Lecture Map

1. Attention
   - What is Attention?
   - Queries, Keys, and Values
   - Attention Steps
   - Training and Inference
   - Multi-Head Attention
   - Summary
2. Transformers
   - Why Attention Alone Isn't Enough
   - The Four Steps
   - The Transformer Block
   - Telling the Model About Time and Position
   - Vision Transformers for Camera Data
   - Transformers for Radar + Camera Fusion
   - Summary
3. Fusion and Uncertainty
   - Probabilistic and Bayesian Attention
   - Association

You Already Know What Attention Is!



*The car pays **most attention** to the pedestrian and less to irrelevant objects.*

Attention in AI works the same way:

When a self-driving car sees many objects, it learns to **focus more** on dangerous ones and **ignore** irrelevant background.

Why Do Sensors Need Attention?

Problem: Too much data, not all useful!

A self-driving car has **multiple sensors**:

Radar	Detects objects in fog & rain but can't tell a car from a person
Camera	Sees colors, shapes, text but fails in darkness & fog

Question: when it's foggy, which sensor should the car trust more?

→ **Radar!** The camera is blinded.
→ Attention learns this *automatically*.

Sunny day

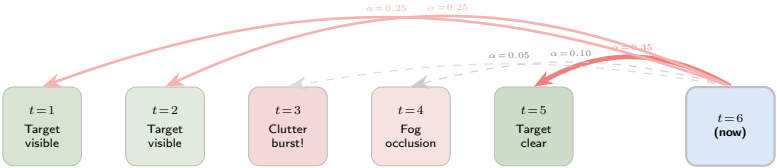
Cameraweight: **70%** | Radarweight: **30%**

Foggy night

Cameraweight: **15%** | Radarweight: **85%**

*Attention **automatically shifts** weights based on conditions!*

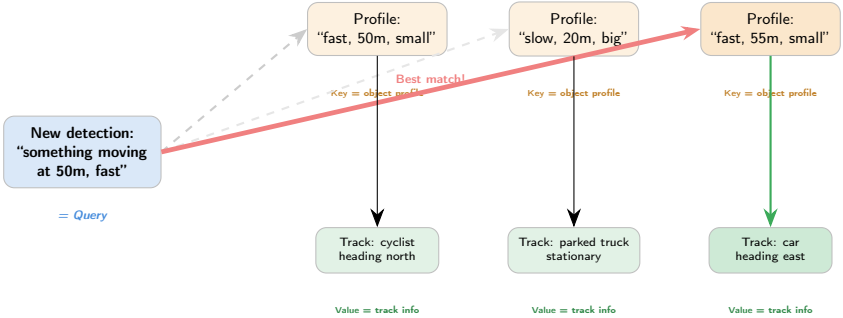
Not All Frames Are Equal Either!



The current frame **pays most attention** to clear past frames ($t = 5, 1, 2$) and **ignores** clutter-corrupted frames ($t = 3, 4$).

This is **temporal attention** — focusing on the most informative moments in time.

The Security Camera Analogy

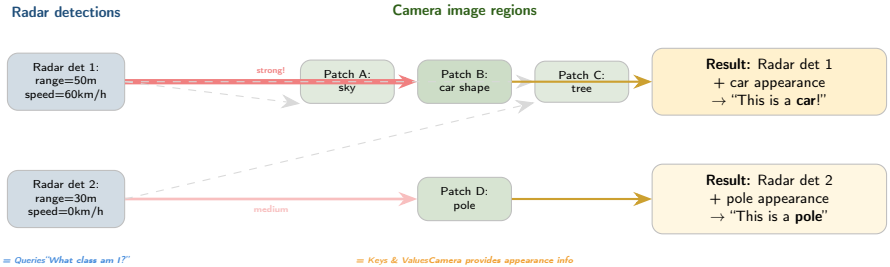


Query = new detection
What did the sensor just see?

Key = object profile
What does each tracked object look like?

Value = track info
What do we know about that object?

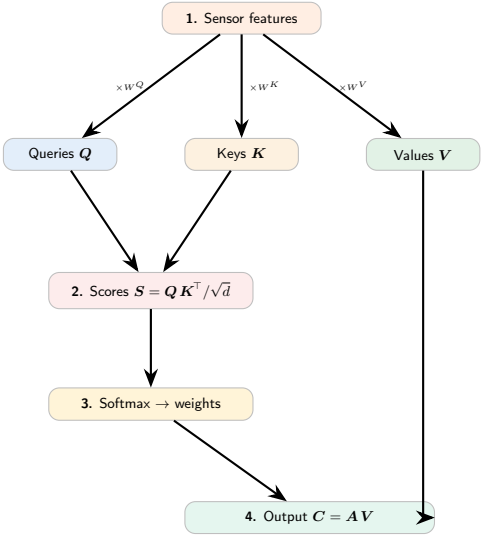
Cross-Attention: Radar Asks the Camera for Help



Cross-attention = one sensor asks another for help

Radar knows *where* and *how fast*, but not *what*. **Camera** knows *what* things look like. Cross-attention lets radar **query** the camera to get appearance information!

The Self Attention



The Attention Formula:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{Q \cdot K^T}{\sqrt{d}}\right) \times V$$

Same formula whether the input is radar data, camera frames, or text!

Step 1: Represent Sensor Data as Numbers

Sensors already produce numbers! We just need to organize them into **feature vectors**:

Radar det 1 → $[50,\ 60,\ 0.8,\ 2.1]$
range, speed, RCS, azimuth

Radar det 2 → $[30,\ 0,\ 0.3,\ -1.5]$
range, speed, RCS, azimuth

Camera patch → $[0.7,\ 0.2,\ 0.9,\ 0.1]$
from a neural network encoder

Key insight

A neural network **encoder** converts raw sensor data into a compact feature vector ("embedding").

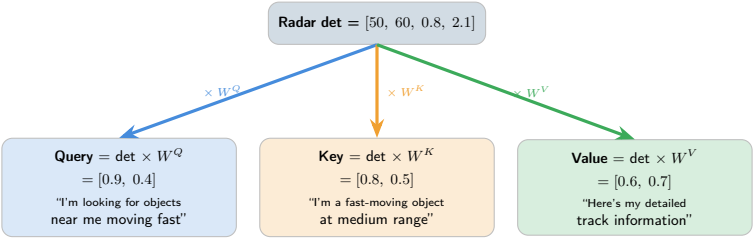
Radar: features might include range, speed, radar cross-section.

Camera: a CNN extracts visual features like edges, shapes, colors into a vector.

Similar objects → similar vectors!

Step 2: Create Queries, Keys, and Values

Each detection gets transformed into three different roles:



W^Q, W^K, W^V are **weight matrices** — tables of numbers that the computer **learns during training**. They transform the same detection into three different "views": what it's looking for, how it describes itself, and what info it carries.

Step 3: Compute Attention Scores

How relevant is each camera patch to my radar detection?

We use the **dot product**:

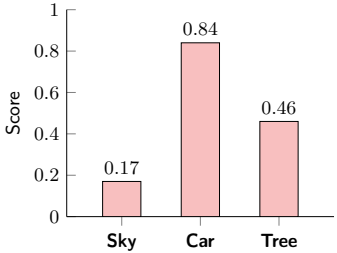
Radar Query $= [0.9,\ 0.4]$

Camera Key A (sky) $= [0.1, 0.2]$ $\rightarrow 0.09 + 0.08 = \mathbf{0.17}$

Camera Key B (car) $= [0.8, 0.3]$ $\rightarrow 0.72 + 0.12 = \mathbf{0.84}$

Camera Key C (tree) $= [0.2, 0.7]$ $\rightarrow 0.18 + 0.28 = \mathbf{0.46}$

Highest score = most relevant!
The car-shaped patch matches the radar detection best $\rightarrow$ this radar point is likely a car.



The dot product measures how well two feature vectors "match."

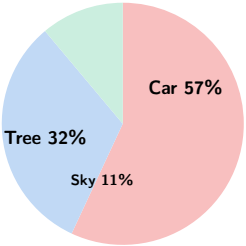
Step 4: Softmax — Turn Scores into Percentages

We want **weights that add up to 100%**.

Softmax converts scores to probabilities:

Sky (patch A):	0.17	$\rightarrow$ **11%**
Car (patch B):	0.84	$\rightarrow$ **57%**
Tree (patch C):	0.46	$\rightarrow$ **32%**
Total:		**100%**

Analogy: You have 100 coins of "trust" to distribute among camera patches. You give the most coins to the patch that best matches your radar detection.



The radar detection focuses most attention on the "car" camera patch

Step 5: Weighted Sum — Combine Information

Finally: mix the camera Values using attention weights.

Value of "sky" $= [0.1,\ 0.0,\ 0.2]$	$\times\ 0.11$	$= [0.01,\ 0.00,\ 0.02]$
Value of "car" $= [0.9,\ 0.8,\ 0.7]$	$\times\ 0.57$	$= [0.51,\ 0.46,\ 0.40]$
Value of "tree" $= [0.3,\ 0.1,\ 0.4]$	$\times\ 0.32$	$= [0.10,\ 0.03,\ 0.13]$

Sum $= [0.62,\ 0.49,\ 0.55]$ → Radar detection enriched with camera appearance → "It's a **car**!"

What just happened?

The radar detection combined visual information from all camera patches, but **mostly from the car-shaped patch** (57% weight). Now the radar "knows" it's looking at a car — something radar alone couldn't determine!

Attention as Geometry in Feature Space

Setup:
Camera patches and the fused radar–camera representation are vectors in a learned feature space ($D \gg 3$).

Each point represents:

- a patch embedding (Sky, Car, Tree)
- or the attention-weighted sum:

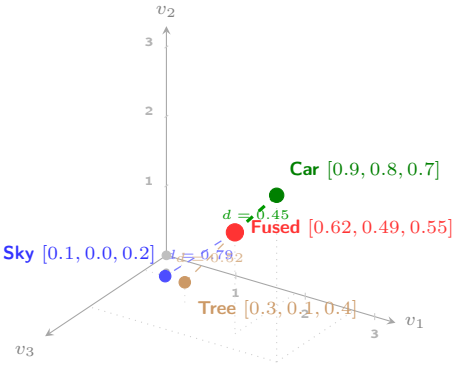
$$\boldsymbol{c} = \sum_j \alpha_j \, \boldsymbol{v}_j$$

Coordinates come directly from Slide 1:

Sky	$= [0.1,\ 0.0,\ 0.2]$	$\alpha = 0.11$
Car	$= [0.9,\ 0.8,\ 0.7]$	$\alpha = 0.57$
Tree	$= [0.3,\ 0.1,\ 0.4]$	$\alpha = 0.32$
Fused	$= [0.62,\ 0.49,\ 0.55]$	—

Key observation:

- Fused lies **closest to Car** (dist $= 0.45$)
- Then Tree (dist $= 0.52$), then Sky (dist $= 0.79$)
- Distance $\Rightarrow$ semantic similarity



Vectors plotted at exact coordinates from Slide 1 (scaled to unit cube $\times 3$)

Self-Attention Layer

Inputs:

Input vectors: $X \quad [N \times d_{\text{in}}]$
Key matrix: $W^K \quad [d_{\text{in}} \times d]$
Value matrix: $W^V \quad [d_{\text{in}} \times d]$
Query matrix: $W^Q \quad [d_{\text{in}} \times d]$

Each **input** produces one **output**, a weighted mix of information from all **inputs**.

Computation:

Queries: $\boldsymbol{Q} = X\,W^Q \quad [N \times d]$
Keys: $\boldsymbol{K} = X\,W^K \quad [N \times d]$
Values: $\boldsymbol{V} = X\,W^V \quad [N \times d]$

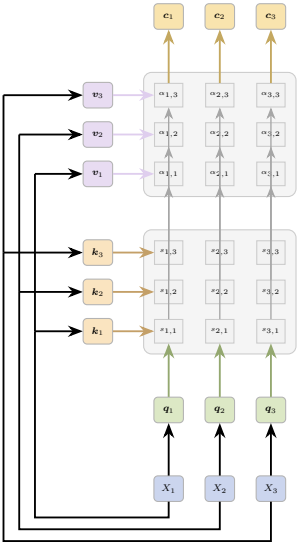
Scores: $\boldsymbol{S} = \boldsymbol{Q}\,\boldsymbol{K}^\top/\sqrt{d} \quad [N \times N]$

$$s_{i,j} = \boldsymbol{q}_i \cdot \boldsymbol{k}_j \,/\sqrt{d}$$

Attention: $\boldsymbol{A} = \text{softmax}(\boldsymbol{S},\ \text{dim}=1)$
$[N \times N]$

Output: $\boldsymbol{C} = \boldsymbol{A}\,\boldsymbol{V} \quad [N \times d]$

$$\boldsymbol{c}_i = \sum \alpha_{i,j}\,\boldsymbol{v}_j$$



Product(→), Sum(↑)

Softmax(↑)

Shapes get simpler:
- N input vectors, each d_{in}
- Almost always $d = d_Q = d_V$

Cross-Attention Layer

Inputs:

Modality A: $X^{(A)}$ $[N_A \times d_A]$
Modality B: $X^{(B)}$ $[N_B \times d_B]$

Modality A provides **queries** (what to look for).

Modality B provides **keys** and **values** (what it contains and what it gives).

Projections:

Queries: $Q = X^{(A)} W^Q$ $[N_A \times d]$
Keys: $K = X^{(B)} W^K$ $[N_B \times d]$
Values: $V = X^{(B)} W^V$ $[N_B \times d]$

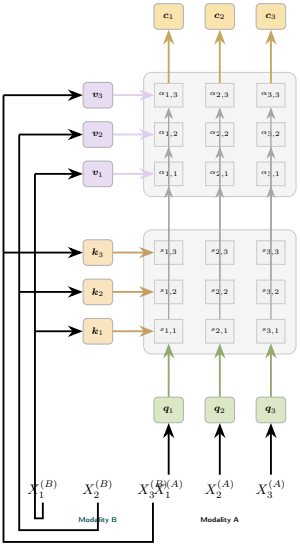
Scores: $S = Q K^\top / \sqrt{d}$ $[N_A \times N_B]$

$$s_{i,j} = \boldsymbol{q}_i \cdot \boldsymbol{k}_j / \sqrt{d}$$

Attention: $A = \mathrm{softmax}(S,\ \mathrm{dim}=2)$ $[N_A \times N_B]$

Output: $O = A V$ $[N_A \times d]$

$$\boldsymbol{c}_i = \sum_j \alpha_{i,j} \boldsymbol{v}_j$$



Product(→), Sum(↑)

Softmax(↑)

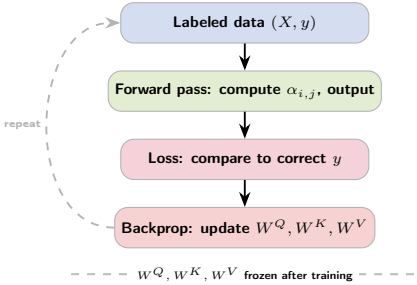
Key insight:
- $X^{(A)}$ → queries only
- $X^{(B)}$ → keys *and* values
- S is $[N_A \times N_B]$, not square

Training vs. Inference — What Happens When?

During Training **(learning phase)**

1. Feed labeled examples (sentence pairs, or radar frames with known targets)
2. Forward pass: compute $\boldsymbol{Q}, \boldsymbol{K}, \boldsymbol{V} \to \alpha_{i,j} \to$ output
3. Compute **loss** by comparing output to correct answer
4. **Backpropagate**: nudge W^Q, W^K, W^V so attention focuses better next time
5. Repeat millions of times

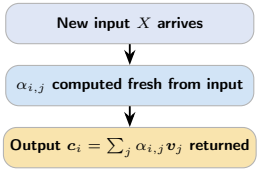
*After training, W^Q, W^K, W^V are **frozen**.*



During Inference **(deployment phase)**

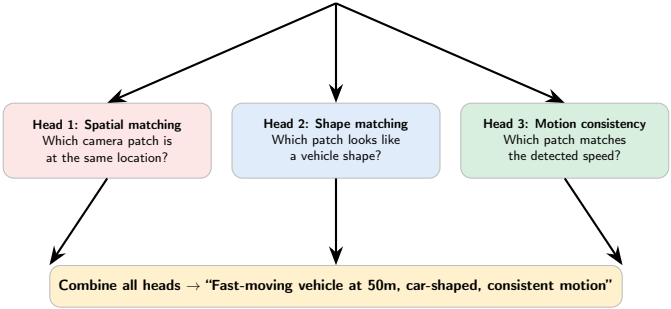
1. A new, unseen input X arrives
2. Compute Q, K, V using the **frozen** W matrices
3. Compute $\alpha_{i,j}$ **fresh** from this specific input
4. Return $\boldsymbol{c}_i = \sum_j \alpha_{i,j} \boldsymbol{v}_j$

*W matrices: **fixed**. Attention weights: **dynamic**.*



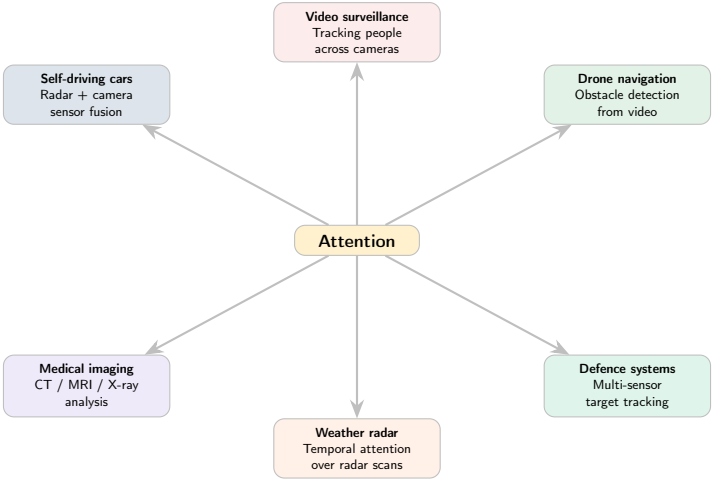
Multi-Head Attention: Multiple Perspectives

Radar detection: "something at 50m, 60km/h"



*Each head has its **own** W^Q, W^K, W^V. Head 1 might learn spatial alignment, Head 2 learns appearance, Head 3 learns motion. Combined, they provide a **complete picture**.*

Attention Is Everywhere!



Key Takeaways

1. **Attention** lets a system focus on the most relevant sensor data, just like a driver focuses on a pedestrian crossing.
2. The core idea: **Query** (what am I looking for?), **Key** (what do I look like?), **Value** (what info do I carry?).
3. **Cross-attention** lets one sensor (radar) ask another (camera) for help — combining "where & how fast" with "what does it look like."
4. The weight matrices W^Q, W^K, W^V are **learned from data** during training. At inference, they are frozen and reused.
5. **Multi-head attention** looks from multiple perspectives: spatial, appearance, motion.

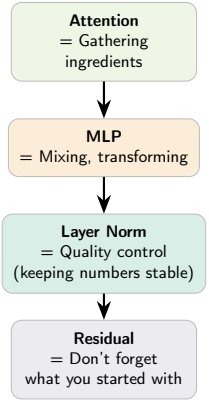
Attention Is Powerful, But It Needs Friends

What attention does well:

- Finds relevant information
- Combines features from different sources
- Works on any number of inputs

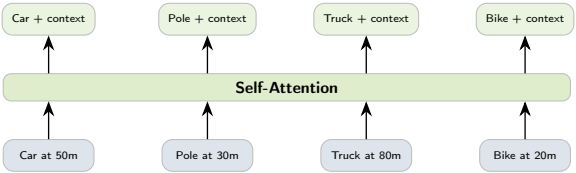
What attention can't do alone:

- No **non-linear processing** after combining
- No **stable training** (numbers can explode!)
- No **memory** of the original input
- Just one pass isn't deep enough



*These 4 ingredients make a **Transformer block**!*

Step 1: Self-Attention



What happened?

Each detection "talked" to every other detection. Now "Car at 50m" knows there's a truck behind it and a bike nearby. Each detection gained **context** about the whole scene.

*Self-attention is the **only step** where detections interact with each other. Everything else works on each detection independently.*

Step 2: MLP (The Thinker)

After attention, each token i has a fused context vector

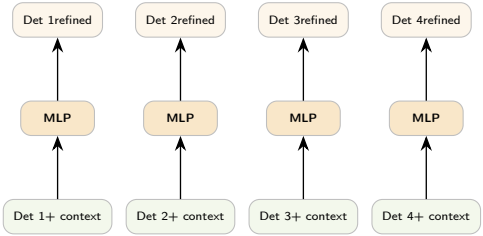
$$\boldsymbol{c}_i = \sum_j \alpha_{i,j}\, \boldsymbol{v}_j.$$

The **multi-layer perceptron (MLP)** processes this vector **individually for each token**.

What does the MLP do?

- Takes c_i for one detection or object
- Applies nonlinear transformations
- Refines latent features and combinations
- Interprets gathered evidence locally

Interpretation: attention gathers information; the MLP reasons about what that information means for one token.



No arrows between columns!
Each MLP works alone.

Important:

- The MLP *does not mix tokens*
- All cross-token interaction already happened in attention

Steps 3: Residual Connections

Problem: after attention and MLP, we might lose the original detection info.

Solution: add the original input back through a skip connection: a direct pathway that bypasses a transformation and adds the original input back to the output, ensuring information and gradients can flow unchanged if needed.

$$\text{output} = \text{MLP}(\text{input}) \; + \; \text{input}$$

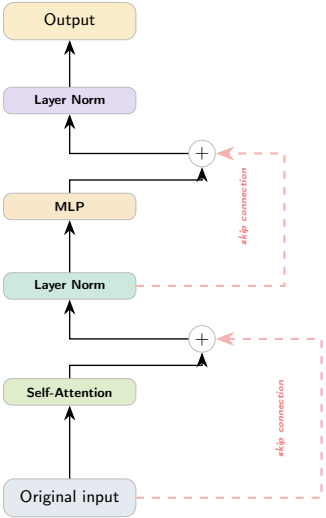
Analogy:
You're writing a book report.

- You read reviews (= attention)
- You form your own thoughts (= MLP)
- But you also keep the original book! (= residual)

Why this helps:

- Original information is never lost
- The MLP learns only a refinement
- Gradients propagate reliably through depth

Interpretation: The network can always fall back to the original representation if the refinement is unhelpful.



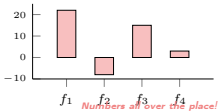
Step 4: Layer Normalization (Quality Control)

Problem: after many additions and multiplications, numbers can get **very big** or **very small**. This makes training unstable.

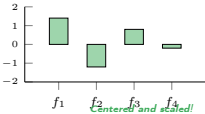
Solution: Layer Normalization rescales numbers to have **zero mean** and **unit variance**.

Think of it like adjusting the **volume knob** on a stereo — keep the signal in a useful range, not too loud, not too quiet.

Before Layer Norm:



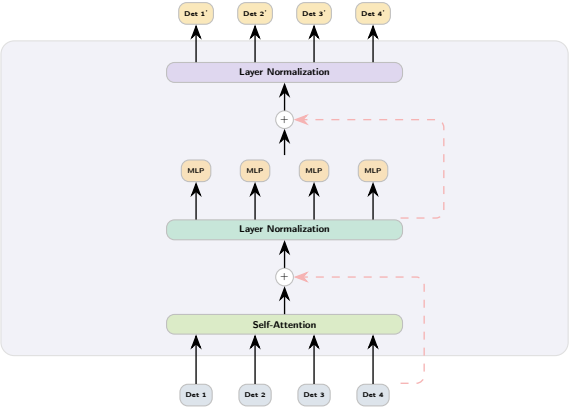
After Layer Norm:



When is it applied?

After each residual addition:
after self-attention + skip,
and after MLP + skip.

Putting It All Together: The Transformer Block



Reading bottom to top:

1. Self-Attention

Every detection talks to every other detection

+ Skip connection

Add original input back (dashed line)

2. Layer Norm

Stabilize numbers

3. MLP (one per detection)

Each detection thinks independently

+ Skip connection

Add pre-MLP signal back

4. Layer Norm

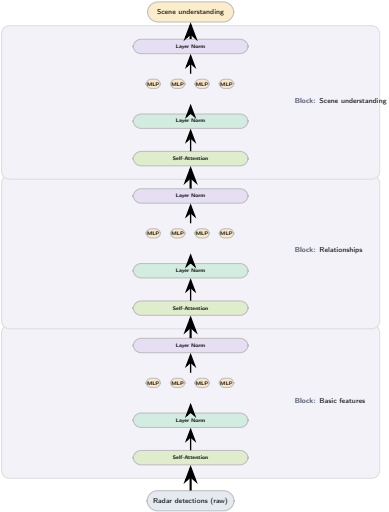
Stabilize again

Stack Blocks = Deeper Understanding

One block gives a basic understanding. **Multiple blocks** give deeper understanding:

- **Block 1:** "I see a fast-moving object nearby"
- **Block 2:** "That object is in the same lane as me, and there's another one behind it"
- **Block 3:** "This looks like a car overtaking, I should slow down"

Each block refines the detections further. Stacking blocks is like thinking about a problem **multiple times** — each pass adds insight.



How Big Are Real Transformers?

Model	Blocks	Embedding size d	Parameters
Small sensor model	4	128	∼2M
PointPillars + Transformer	6	256	∼10M
BEVFormer (self-driving)	6	256	∼68M
Vision Transformer (ViT)	12	768	∼86M
GPT-2	48	1600	1.5B
GPT-4 (estimated)	∼120	∼12000	∼1.7T

Key insight

The **same architecture** (stack of transformer blocks) is used for tiny sensor fusion models on a car's computer AND massive language models in data centers. The only difference is **size**.

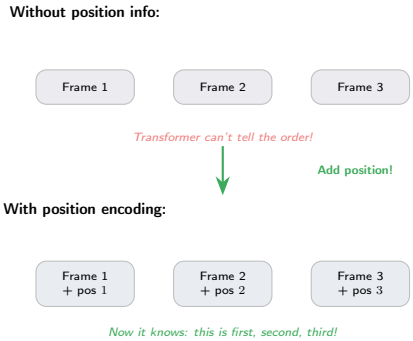
Problem: The Transformer Doesn't Know Order!

Self-attention treats all inputs as a **set**: it doesn't know which detection came first.

But order matters!

- For video: frame 1 $\rightarrow$ frame 2 $\rightarrow$ frame 3 is *very* different from frame 3 $\rightarrow$ frame 1 $\rightarrow$ frame 2
- For radar scans: time matters for tracking
- For camera patches: spatial position matters

Solution: Positional Encoding
Add a unique "position tag" to each input *before* feeding it to the transformer.



Positional Encoding: Injecting Order Information

Self-attention is permutation-invariant: it does not know the order or timing of tokens.

Positional encoding fixes this by assigning each position t a deterministic vector $_t$ and adding it to the input embedding.

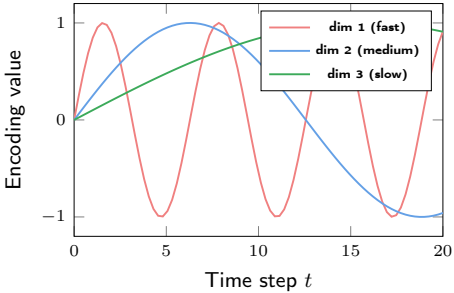
A common choice uses **sine and cosine functions**:

$$p_{t,2i} = \sin\left(\frac{t}{10000^{2i/d}}\right),$$

$$p_{t,2i+1} = \cos\left(\frac{t}{10000^{2i/d}}\right).$$

Key ideas:

- Each position t gets a unique vector pos_t
- Nearby positions have similar encodings
- Relative position offsets can be inferred from the encodings



Different dimensions oscillate at different frequencies, together forming a structured positional signature.

Temporal Self-Attention: Mathematical Formulation

Sequence of sensor feature embeddings $\{\boldsymbol{e}_1, \ldots, \boldsymbol{e}_T\} \in \mathbb{R}^d$ with positional encoding $\boldsymbol{p}_t$:

$$\boldsymbol{q}_t = (\boldsymbol{e}_t + \boldsymbol{p}_t)\, W^Q, \quad \boldsymbol{k}_\tau = (\boldsymbol{e}_\tau + \boldsymbol{p}_\tau)\, W^K, \quad \boldsymbol{v}_\tau = \boldsymbol{e}_\tau\, W^V$$

$$\alpha_{t,\tau} = \frac{\exp\!\big(\boldsymbol{q}_t^\top \boldsymbol{k}_\tau / \sqrt{d}\big)}{\sum_{\tau'=1}^{T} \exp\!\big(\boldsymbol{q}_t^\top \boldsymbol{k}_{\tau'} / \sqrt{d}\big)}, \qquad \hat{\boldsymbol{e}}_t = \sum_{\tau=1}^{T} \alpha_{t,\tau}\, \boldsymbol{v}_\tau$$

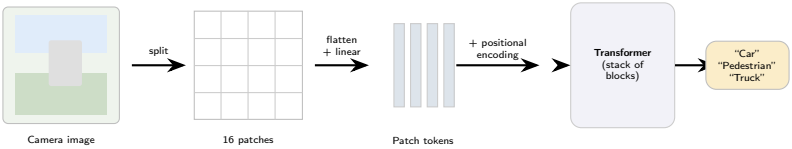
Sinusoidal positional encoding (Vaswani et al.):

$$p_{t,2i} = \sin\!\left(\frac{t}{10000^{2i/d}}\right), \quad p_{t,2i+1} = \cos\!\left(\frac{t}{10000^{2i/d}}\right)$$

Causal masking (real-time): set $\alpha_{t,\tau} = 0$ for $\tau > t$.
Non-causal (offline): attend to full sequence (track smoothing).

Key property: $\boldsymbol{q}_t^\top \boldsymbol{k}_\tau$ decomposes into *content–content*, *content–position*, *position–content*, and *position–position* terms — each captures different temporal structure.

Vision Transformer: Camera Image → Patches → Transformer



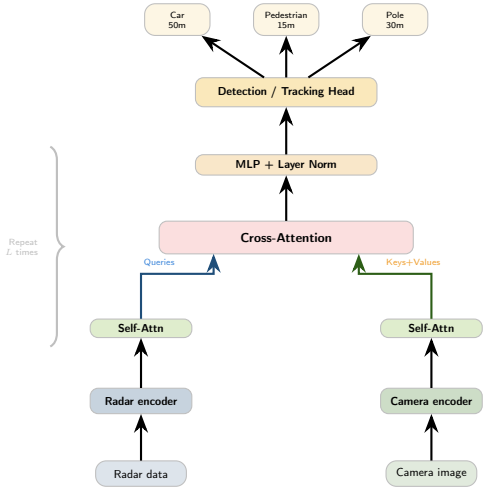
Key ideas

- Split image into patches (like puzzle pieces)
- Each patch becomes a token vector
- Transformer processes all patches together
- Self-attention: every patch can look at every other patch

Why patches?

- A camera image has millions of pixels
- Processing every pixel pair: way too expensive!
- Patches reduce 224×224 pixels to just 196 tokens
- Much more manageable for attention

The Full Picture: Transformer Sensor Fusion



Data flow:

1. Encode each sensor independently

2. Self-attention within each modality (radar talks to radar, camera to camera)

3. Cross-attention: radar queries camera "What does my object look like?"

4. MLP refines the fused features

5. Detection head outputs objects with class + position

The fusion happens at cross-attention!

Radar provides "where & how fast." Camera provides "what it looks like."

What Each Part Does — In Plain English

Component	What it does
Radar encoder	Turns raw radar data (range, speed, angle) into feature vectors
Camera encoder	Turns camera pixels into feature vectors (using CNN or ViT)
Self-attention (radar)	Each radar detection learns about the full radar scene
Self-attention (camera)	Each image patch learns about the full image
Cross-attention	Radar detections ask camera patches for appearance info; this is where the two sensors **fuse**
MLP + LayerNorm	Refine the fused features; keep numbers stable
Detection head	Convert features into final outputs: object class, position, speed

The magic of cross-attention

A Self-Driving Car in Action

What happens in 50 milliseconds:

1. Radar scan
6 detections
at various ranges

→

2. Camera frame
1920×1080 pixels
split into patches

→

3. Transformer
6 blocks process
tokens in parallel

→

4. Output
3 objects detected
with class + position

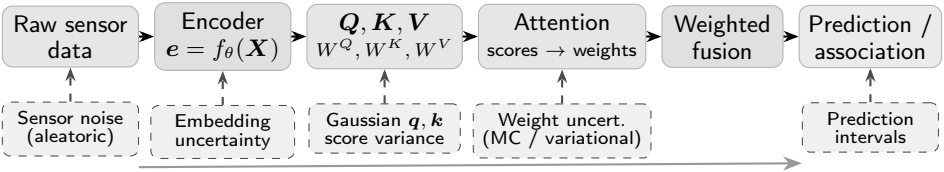
Detection 1: Car, 50m ahead, 60km/h, same lane → safe distance
Detection 2: Pedestrian, 15m, crossing → **BRAKE!**
Detection 3: Pole, 30m, stationary → no threat

The transformer combined radar (where & how fast) with camera (what it looks like) to make life-saving decisions — all in less than 50 milliseconds!

What We Learned Today

1. A **Transformer block** has 4 ingredients: Self-Attention, MLP, Residual Connections, and Layer Norm.
2. **Stacking blocks** gives deeper understanding: block 1 sees features, block 2 sees relationships, block 3 understands the scene.
3. **Positional encoding** tells the model about time (video frames) and space (image patches, radar angles).
4. **Vision Transformers** split images into patches and process them with standard transformer blocks.
5. **Cross-attention** is how radar and camera fuse: one sensor *queries* the other, combining "where" with "what."
6. The **same architecture** scales from tiny sensor models to GPT-4.

Where Uncertainty Enters the Attention Pipeline



uncertainty propagates and accumulates through the pipeline

Sources of uncertainty:

- **Sensor noise** (aleatoric): radar multipath, camera blur, fog, low SNR — present *before* any learning
- **Embedding uncertainty**: encoder maps noisy inputs to uncertain feature vectors $e \sim \mathcal{N}(\mu_e, \Sigma_e)$
- **Score variance**: if q or k are uncertain, the dot product $q^\top k$ has non-zero variance → uncertain attention weights

How to handle it:

- **Aleatoric**: heteroscedastic outputs ($\sigma^2(x)$ predicted by the network), sensor-level noise models
- **Epistemic**: MC dropout, deep ensembles, variational Bayes over W^Q, W^K, W^V
- **Output**: prediction intervals, calibrated confidence scores, conformal prediction for coverage guarantees

Why Standard Attention Fails Uncertain Sensors

Problem: Softmax attention only sees *mean* embeddings — completely blind to how *reliable* those embeddings are.

Standard (deterministic) attention:

$$\alpha_j = \mathrm{softmax}\left(\frac{\boldsymbol{\mu}_q^\top \boldsymbol{\mu}_{k_j}}{\sqrt{d}}\right)$$

- Uses only the **mean** embedding vector
- Two keys with identical mean scores get identical weights
- A foggy camera = a clear radar, if their means match
- Overconfident in safety-critical fusion

What we want instead:

- Sensor with high uncertainty Σ_{k_j}
  $\Rightarrow$ *lower* attention weight
- Sensor with low uncertainty
  $\Rightarrow$ *higher* attention weight
- Analogous to Kalman gain:
  $\boldsymbol{K} \propto \boldsymbol{P}\boldsymbol{H}^\top \boldsymbol{S}^{-1}$ (covariance-weighted)

Solution: Model each embedding as a Gaussian random variable and propagate uncertainty through softmax via Monte Carlo sampling.

Gaussian Embeddings: Derivation of Score Statistics

Model queries and keys as Gaussian random vectors:

$$\boldsymbol{q} \sim \mathcal{N}(\boldsymbol{\mu}_q,\ \boldsymbol{\Sigma}_q), \qquad \boldsymbol{k}_j \sim \mathcal{N}(\boldsymbol{\mu}_{k_j},\ \boldsymbol{\Sigma}_{k_j}), \qquad \text{independent}$$

Score: $s_j = \boldsymbol{q}^\top \boldsymbol{k}_j / \sqrt{d}$. Since s_j is a bilinear form in Gaussian vectors:

$$\mathbb{E}[s_j] = \frac{\boldsymbol{\mu}_q^\top \boldsymbol{\mu}_{k_j}}{\sqrt{d}}$$

$$\mathrm{Var}(s_j) = \frac{1}{d}\Big(\boldsymbol{\mu}_q^\top \boldsymbol{\Sigma}_{k_j} \boldsymbol{\mu}_q + \boldsymbol{\mu}_{k_j}^\top \boldsymbol{\Sigma}_q \boldsymbol{\mu}_{k_j} + \mathrm{tr}(\boldsymbol{\Sigma}_q \boldsymbol{\Sigma}_{k_j})\Big)$$

Proof sketch: Write $s_j = \sum_i q_i k_{j,i}/\sqrt{d}$. Each product $q_i k_{j,i}$ has mean $\mu_{q,i}\mu_{k_j,i}$ and variance $\mu_{q,i}^2 \sigma_{k_j,i}^2 + \mu_{k_j,i}^2 \sigma_{q,i}^2 + \sigma_{q,i}^2 \sigma_{k_j,i}^2$ (for independent q_i, $k_{j,i}$). Sum over i and collect terms.

Key insight: high $\mathrm{Var}(s_j) \Rightarrow$ low confidence that $\boldsymbol{k}_j$ truly matches $\boldsymbol{q}$.

Uncertainty-Aware Attention Mechanisms

Approach 1: Adaptive temperature. Weight the logit by its own uncertainty:

$$\alpha_j \propto \exp\left(\frac{\mathbb{E}[s_j]}{\sqrt{\mathrm{Var}(s_j) + \epsilon}}\right)$$

Interpretation: uncertain scores are scaled down before softmax.

Approach 2: Monte Carlo Bayesian attention.

$$\mathbb{E}[\alpha_j] \approx \frac{1}{S}\sum_{s=1}^{S} \mathrm{softmax}\left(\frac{\boldsymbol{q}^{(s)\top}\boldsymbol{k}_j^{(s)}}{\sqrt{d}}\right), \qquad \boldsymbol{q}^{(s)} \sim \mathcal{N}(\boldsymbol{\mu}_q, \boldsymbol{\Sigma}_q)$$

Approach 3: Variational Bayesian attention. Place a posterior $q(\boldsymbol{\theta})$ over projection weights; marginalize: $p(\alpha \mid X) = \int \mathrm{softmax}(f_\theta(X))\, q(\theta)\, d\theta$.

Approach 4: MC Dropout. At inference, run S forward passes with dropout active. Variance of attention weights $\approx$ epistemic uncertainty.

Example of Object Detection: Understanding the Setup

The physical scenario:

- An autonomous vehicle has two sensors pointing at the *same* object
- **Radar** — reliable in any weather, but low resolution
- **Camera** — high detail, but today there is fog
- Both return a **feature vector** (compact numerical description of the detected object: size, velocity, shape, position...)
- Task: *combine* both reports into one fused decision

What is the query q?

- The query is the **current state estimate** of the tracked object — what the fusion system *expects* to see based on prior frames
- It asks: "which sensor report best matches what I already know about this target?"
- Keys $\boldsymbol{k}_j$ are the sensor embeddings being evaluated against that expectation

Where does Σ_{k_j} (variance) come from?

In a real system it is *not* hand-picked. It is estimated by one of:

1. **Offline calibration:** record many measurements of a known target under controlled conditions; compute empirical covariance of the resulting embeddings
2. **Physical sensor model:** radar range uncertainty $\propto$ distance^2; camera uncertainty $\propto$ noise model of optics
3. **Learned from data:** a neural encoder outputs both $\boldsymbol{\mu}_{k_j}$ and Σ_{k_j} (heteroscedastic regression)
4. **Online estimation:** track variance of recent embeddings from the same sensor in a sliding window

In this notebook $\Sigma_{\text{camera}} = 0.5\,\boldsymbol{I}$ is chosen *by hand* to create a clear pedagogical example. In deployment it would come from methods 1–4 above.

The Core Insight: Why Similar Means Are Not Enough

Why are the two embeddings nearly identical?

Both sensors are looking at the *same physical object*. A car at 50 m travelling at 60 km/h produces similar feature values regardless of which sensor reports it:

	Radar	Camera (fog)
Mean $\boldsymbol{\mu}_{k_j}$	$[0.95, 0.45, \ldots]$	$[0.90, 0.40, \ldots]$
Variance σ^2	0.01 (tight)	0.50 (wide)
Score $\mathbb{E}[s_j]$	0.648	0.604

This is the hardest and most important case:

- If scores were *very different* $\Rightarrow$ deterministic attention already works fine
- Similar scores but *different reliability* $\Rightarrow$ deterministic attention fails; Bayesian attention is needed
- This is **deliberate** — it stress-tests the method by removing the easy signal

[colback=red!8,colframe=red!60,boxrule=0.6pt] **The fundamental problem:**

Two sensors agreeing *on average* tells you nothing if one of them is guessing randomly half the time.

The mean is the same. The *trustworthiness* is not.

Analogy — Kalman filter:
The Kalman gain $\boldsymbol{K} = \boldsymbol{P}\boldsymbol{H}^\top \boldsymbol{S}^{-1}$ does not just look at which measurement is closest to the prediction. It weights by *covariance* $\boldsymbol{S}$. A noisy measurement that happens to be close is trusted *less* than a reliable measurement that is equally close.

Bayesian attention is the transformer analogue of the Kalman gain: it incorporates covariance into the weighting, not just the mean.

Experimental Setup: The Fog Bank Scenario

1 query, 4 keys representing sensor modalities ($d = 4$):

Key	Sensor	Mean score $\mathbb{E}[s_j]$	σ^2
1	Radar (clear)	0.648	**0.01**
2	Camera (foggy)	0.604	**0.50**
3	Weak detection	0.265	0.03
4	Different target	0.415	0.02

The trap for deterministic attention:

- Keys 1 & 2 have *nearly identical* mean scores (0.648 vs 0.604 — only 7% difference)
- Yet Key 2 has **50× higher variance**
- Deterministic softmax assigns them almost equal weights
- This is dangerous in real sensor fusion

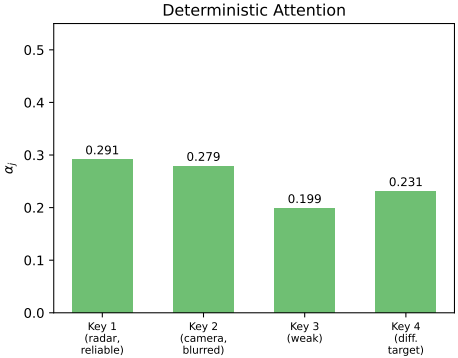
Query: $\boldsymbol{\mu}_q = [1.0,\ 0.5,\ 0.3,\ 0.2],\ \Sigma_q = 0.01\,\boldsymbol{I}$ (confident)

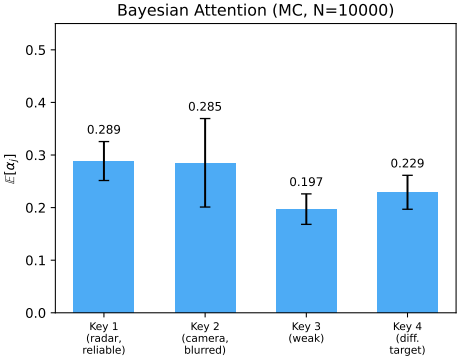
Key 1 (reliable radar):
$\boldsymbol{\mu}_{k_1} = [0.95, 0.45, 0.30, 0.15]$
$\Sigma_{k_1} = 0.01\,\boldsymbol{I}$ (tight)

Key 2 (foggy camera):
$\boldsymbol{\mu}_{k_2} = [0.90, 0.40, 0.28, 0.12]$
$\Sigma_{k_2} = 0.50\,\boldsymbol{I}$ (very spread)

Values: $V = \boldsymbol{I}_4$ (identity)
so output $\equiv$ weight vector directly.

Result 1 — Deterministic vs. Bayesian Weights





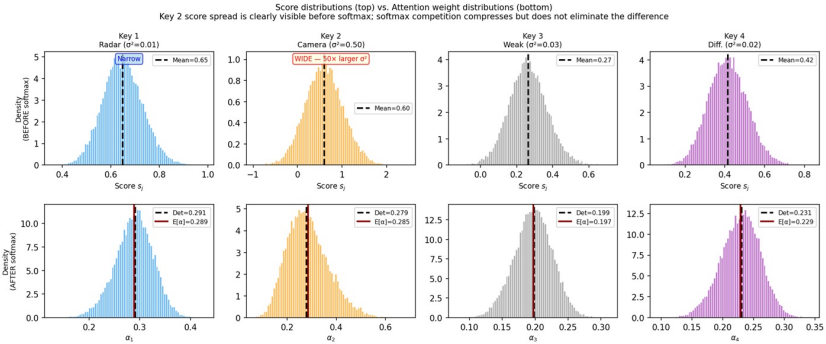
Deterministic attention (left panel):

- Key 1: $\alpha = 0.291$, Key 2: $\alpha = 0.279$
- Nearly equal — cannot distinguish reliable from unreliable
- No information about weight reliability

Bayesian attention (right panel):

- Key 1: $\mathbb{E}[\alpha] = 0.288$, small error bar
- Key 2: $\mathbb{E}[\alpha] = 0.286$, large error bar
- Error bar = $\sqrt{\mathrm{Var}[\alpha_j]}$ across 10 000 MC samples
- **The error bar is the trustworthiness signal**

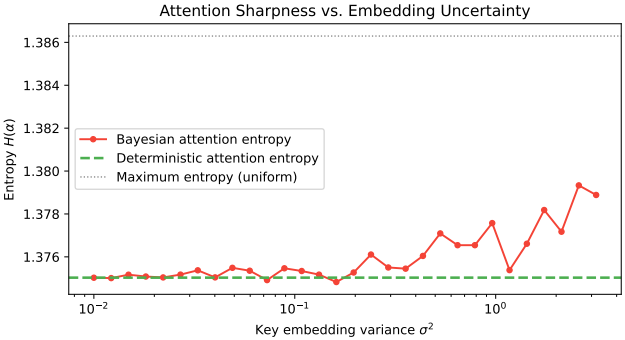
Result 2 — Full Distribution of Attention Weights



- **Top row — score distributions (before softmax):** Key 2 (camera, $\sigma^2{=}0.50$) has a score *spread* $\approx 5\times$ wider than Key 1 (radar, $\sigma^2{=}0.01$). Note: the embedding *variance* ratio is $50\times$, but score *std* scales as $\sqrt{\sigma^2}$, so the visible spread ratio is $\sqrt{50} \approx 7\times$.
- **Bottom row — attention weights (after softmax):** Softmax competition compresses the difference further. Key 2's weight still spans a much wider range (0.07–0.69) compared to Key 1 (0.13–0.43).
- **Key formula:** $\mathrm{Std}(s_j) \approx \sigma_{k_j}\, \|\boldsymbol{\mu}_q\|/\sqrt{d}$ — uncertainty in the embedding propagates as its *square root* into the attention scores.

A score that is sometimes excellent and sometimes terrible is more *dangerous* than one that is consistently mediocre. The

Result 3 — Attention Entropy vs. Sensor Noise



Entropy of the weight distribution:

$$H(\alpha) = -\sum_j \alpha_j \log \alpha_j$$

- $H = 0$: all weight on one key (maximally sharp)
- $H = \log 4 \approx 1.39$: uniform (no preference)

As σ^2 increases:

- Bayesian: entropy rises — model correctly admits it cannot distinguish keys
- Deterministic: entropy flat — always equally overconfident

Summary: What MC Bayesian Attention Gives You

	Deterministic	**MC Bayesian**
Input	$\boldsymbol{\mu}_q$, $\boldsymbol{\mu}_{k_j}$ only	$\boldsymbol{\mu}_q, \Sigma_q$, $\boldsymbol{\mu}_{k_j}, \Sigma_{k_j}$
Output	Single weight α_j	$\mathbb{E}[\alpha_j]$, $\mathrm{Var}[\alpha_j]$, full distribution
Uncertainty	Ignored	Propagated through softmax
Overconfident?	Yes	No
Cost	$O(nd)$	$O(S \cdot nd)$, $S \approx 50$–200 in practice

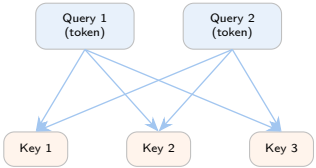
Three outputs of practical value:

1. $\mathbb{E}[\alpha_j]$ — best estimate of attention weight *(use for fusion output)*
2. $\mathrm{Var}[\alpha_j]$ — reliability of that weight *(use for anomaly / fault detection)*
3. Full distribution — enables downstream uncertainty propagation *(use for safety bounds)*

Approach 3 (Variational Bayes) places a posterior over projection weights W^Q, W^K, W^V — handles *epistemic* uncertainty from limited training data, not just sensor noise.

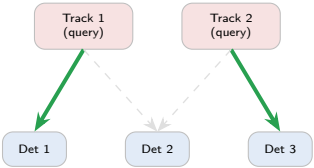
Association *Is* Attention — But With Structure

Standard Attention



Each query attends to all keys.
No constraints on columns.

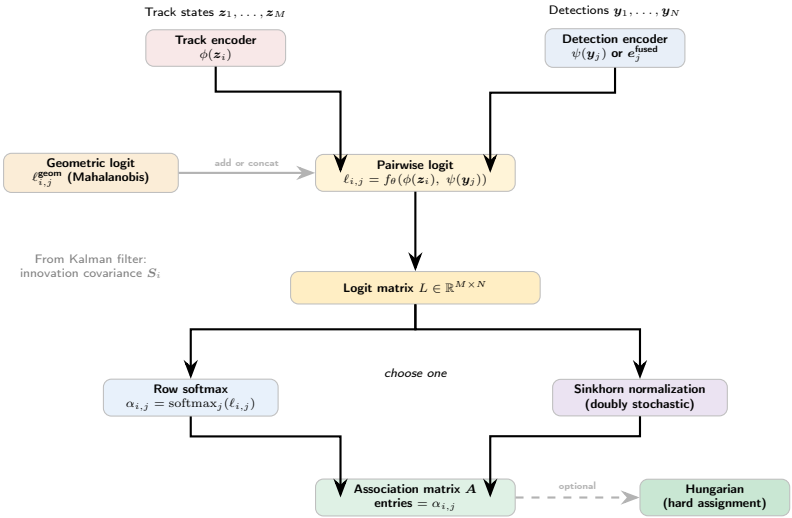
Association as Structured Attention



Each detection assigned
*to **at most one** track.*
Structural constraint enforced.

	Attention	**Association**
Query	token $\boldsymbol{q}_i$	track state $\boldsymbol{z}_i$
Key	token $\boldsymbol{k}_j$	detection $\boldsymbol{y}_j$
Row constraint	$\sum_j \alpha_{i,j} = 1$	$\sum_j \alpha_{i,j} = 1$
Column constraint	none	$\sum_i \alpha_{i,j} \leq 1$
Assignment	soft	soft $\rightarrow$ hard

Architecture of the Association Network



How Logits Are Constructed: Two Sources of Evidence

1. Geometric logit (model-based)

Uses the **predicted measurement** from the Kalman filter:

$$\ell_{i,j}^{\text{geom}} = -\frac{1}{2}\underbrace{(\boldsymbol{y}_j - \boldsymbol{H}\boldsymbol{z}_i)^\top}_{\text{innovation}}\underbrace{\boldsymbol{S}_i^{-1}}_{\text{precision}}(\boldsymbol{y}_j - \boldsymbol{H}\boldsymbol{z}_i)$$

Intuition: this is the negative Mahalanobis distance. Detections close to the predicted position (in covariance-scaled coordinates) get high logits.

Equivalent to: the log-likelihood under a Gaussian measurement model $\boldsymbol{y}_j \sim \mathcal{N}(\boldsymbol{H}\boldsymbol{z}_i, \boldsymbol{S}_i)$.

2. Learned logit (data-driven)

Uses **embeddings** from the fusion network:

$$\ell_{i,j}^{\text{learn}} = f_\theta(\phi(\boldsymbol{z}_i),\ \psi(\boldsymbol{y}_j))$$

where ϕ, ψ are neural network encoders and f_θ can be:

- dot product: $\phi(\boldsymbol{z}_i)^\top \psi(\boldsymbol{y}_j)$
- bilinear: $\phi(\boldsymbol{z}_i)^\top W \psi(\boldsymbol{y}_j)$
- MLP: $\text{MLP}([\phi(\boldsymbol{z}_i); \psi(\boldsymbol{y}_j)])$

Advantage: can incorporate appearance, class, and cross-modal features that geometry alone cannot capture.

In practice: combine both

$\ell_{i,j} = \ell_{i,j}^{\text{geom}} + \ell_{i,j}^{\text{learn}}$. The geometric term provides a strong prior; the learned term adds discriminative power.

Worked Example: 3 Tracks, 4 Detections

Logit matrix (combined geometric + learned):

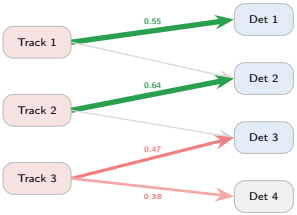
$$L = \begin{bmatrix} 1.8 & 1.4 & -0.4 & -1.0 \\ 0.2 & 2.1 & 1.0 & -0.6 \\ -0.8 & 0.5 & 1.9 & 1.6 \end{bmatrix}$$

Row-wise softmax $\rightarrow$ soft association $\boldsymbol{A}$:

$$\boldsymbol{A} = \begin{bmatrix} 0.55 & 0.37 & 0.06 & 0.02 \\ 0.10 & 0.64 & 0.23 & 0.04 \\ 0.03 & 0.12 & 0.47 & 0.38 \end{bmatrix}$$

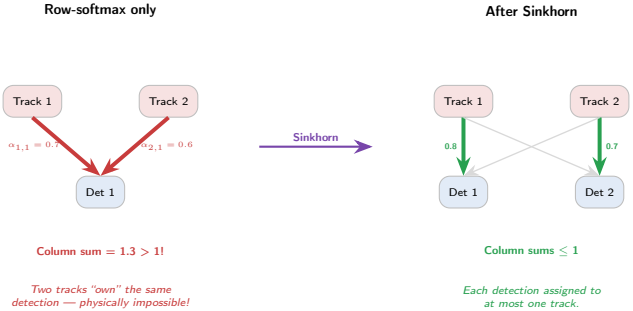
Reading the matrix:

- Track 1 $\rightarrow$ Det 1 (55%, confident)
- Track 2 $\rightarrow$ Det 2 (64%, confident)
- Track 3 $\rightarrow$ Det 3 *or* Det 4 (47% vs 38%)
  $\Rightarrow$ **ambiguous!**



Track 3 is ambiguous — same pattern as JPDA diffuse weights!

The Column Problem: Why Row-Softmax Is Not Enough



This is "structured attention"

Standard attention imposes **row** constraints only ($\sum_j \alpha_{i,j} = 1$). **Structured attention** adds **column** constraints ($\sum_i \alpha_{i,j} \leq 1$), making the weight matrix **doubly stochastic** — a proper soft assignment.

Sinkhorn Normalization: Step by Step

Algorithm: Sinkhorn Association

Algorithm 1 Sinkhorn Normalization

Require: Logit matrix $L \in \mathbb{R}^{M \times N}$, temperature τ, iterations K

$P \leftarrow L/\tau$
for $k = 1, \ldots, K$ **do**
  $P_{ij} \leftarrow P_{ij} - \log \sum_j \exp(P_{ij})$ *// row normalize*
  $P_{ij} \leftarrow P_{ij} - \log \sum_i \exp(P_{ij})$ *// col normalize*
end for
$A \leftarrow \exp(P)$ *// approximately doubly stochastic* **return** A

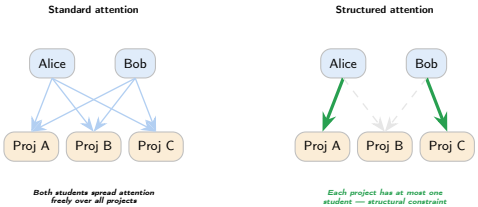
Properties

- Differentiable $\Rightarrow$ trainable end-to-end
- Solves optimal transport
- Hard assignment: Hungarian on $\boldsymbol{A}$

What Is Structured Attention? — The Intuition

Core idea: standard attention lets every query grab as much from every key as it wants. **Structured attention** adds rules about *who can grab what*.

Think of it as assigning students to project groups:



The "structure" is the **rule about columns**: each project (detection) can be claimed by at most one student (track).

Standard (unstructured) attention:

$$\alpha_{i,j} = \frac{\exp(s_{i,j})}{\sum_{j'} \exp(s_{i,j'})}$$

- Each row is independently normalized
- No constraints between rows
- Output: $\boldsymbol{c}_i = \sum_j \alpha_{i,j}\, \boldsymbol{v}_j$

Structured attention replaces softmax with a constrained optimization:

$$\boldsymbol{A}^* = \arg\max_{\boldsymbol{A} \in \mathcal{C}} \sum_{i,j} A_{i,j}\, s_{i,j} - \Omega(\boldsymbol{A})$$

where $\mathcal{C}$ is a constraint set and Ω is a regularizer.

Structured Attention by Example: 2 Tracks, 3 Detections

Same logit matrix L, three different normalizations:

Logit matrix L

$$L = \begin{bmatrix} 2.0 & 0.5 & 0.1 \\ 0.3 & 1.8 & 0.4 \end{bmatrix} \longrightarrow$$

Level 1: Row softmax

$$\begin{bmatrix} 0.74 & 0.17 & 0.11 \\ 0.10 & 0.64 & 0.16 \end{bmatrix} \longrightarrow$$

Rows sum to 1.
Column 1 sums to 0.84
(could be > 1
with more tracks)

Level 2: Sinkhorn

$$\begin{bmatrix} 0.81 & 0.10 & 0.09 \\ 0.06 & 0.72 & 0.22 \end{bmatrix} \longrightarrow$$

Rows and columns
sum to ≈ 1.
Weight pushed toward
one-to-one matching.

Level 3: Hungarian

$$\begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \end{bmatrix}$$

Hard zeros and ones.
Track 1 $\rightarrow$ Det 1.
Track 2 $\rightarrow$ Det 2.
Det 3 is unmatched.

When to use which?

- **Row softmax:** quick, differentiable, but allows double-counting (two tracks claim same detection).
- **Sinkhorn:** differentiable *and* prevents double-counting. Best for end-to-end training.

Practical recipe

1. **Training:** use Sinkhorn (differentiable, enforces structure, gradients flow).
2. **Inference:** run Sinkhorn, then optionally apply Hungarian on the soft matrix to get a hard 0/1 assignment.

Structured Attention vs. Classical Association

Method	Weight source	Structure	Trainable?
Nearest neighbor	min distance	hard, greedy	no
PDAF / JPDA	likelihood $p(\boldsymbol{y}_j \mid \boldsymbol{z}_i)$	soft, row-normalized	no (model-based)
Hungarian	cost matrix	hard, one-to-one	no (combinatorial)
Row-softmax attention	learned logits	soft, row only	yes
Sinkhorn attention	learned logits	soft, doubly stochastic	yes, end-to-end

PDAF connection

PDAF computes $\beta_{i,j} \propto p(\boldsymbol{y}_j \mid \boldsymbol{z}_i)$ and forms $\hat{\boldsymbol{y}}_i = \sum_j \beta_{i,j} \boldsymbol{y}_j$.
This is **exactly** soft attention with likelihood-based logits. Structured attention extends this with learned logits and column constraints.

Modern systems (DETR, TransTrack)

Object queries in transformer detectors are essentially **track queries**. The attention matrix between queries and image features is an implicit association. Bipartite matching in the loss enforces one-to-one during training.

Summary: Association as Structured Attention

1. **Data association** asks: which detection belongs to which track? This is the same question as attention: which items are most relevant to a query?
2. The difference is **structure**: association needs column constraints (each detection $\rightarrow$ at most one track), while standard attention does not.
3. **Logits** combine geometric evidence (Mahalanobis / Kalman innovation) with learned features from the fusion network.
4. **Sinkhorn normalization** turns a logit matrix into a doubly stochastic assignment, bridging soft attention and hard one-to-one matching. It is differentiable and trainable end-to-end.
5. **Structured attention** is the general framework: replace softmax with constrained optimization over a structured set (simplex, Birkhoff polytope, permutations, etc.).
6. In modern architectures (DETR, BEVFormer), fusion and association are increasingly **joint**: the same transformer handles both, with structural constraints in the loss.

Acknowledgements/ Document preparation

This document was converted into LaTeX slides and formatted with assistance from Claude (Anthropic). The instructor verified the final structure and wording.